



Advanced Deep Neural Network Training

Huỳnh Nhật Toàn - DSC

Mục tiêu: Giúp cho các bạn có kiến thức nền tảng về khoa học dữ liệu, hiểu được quy trình và có thể tự tin xây dựng những mô hình ML





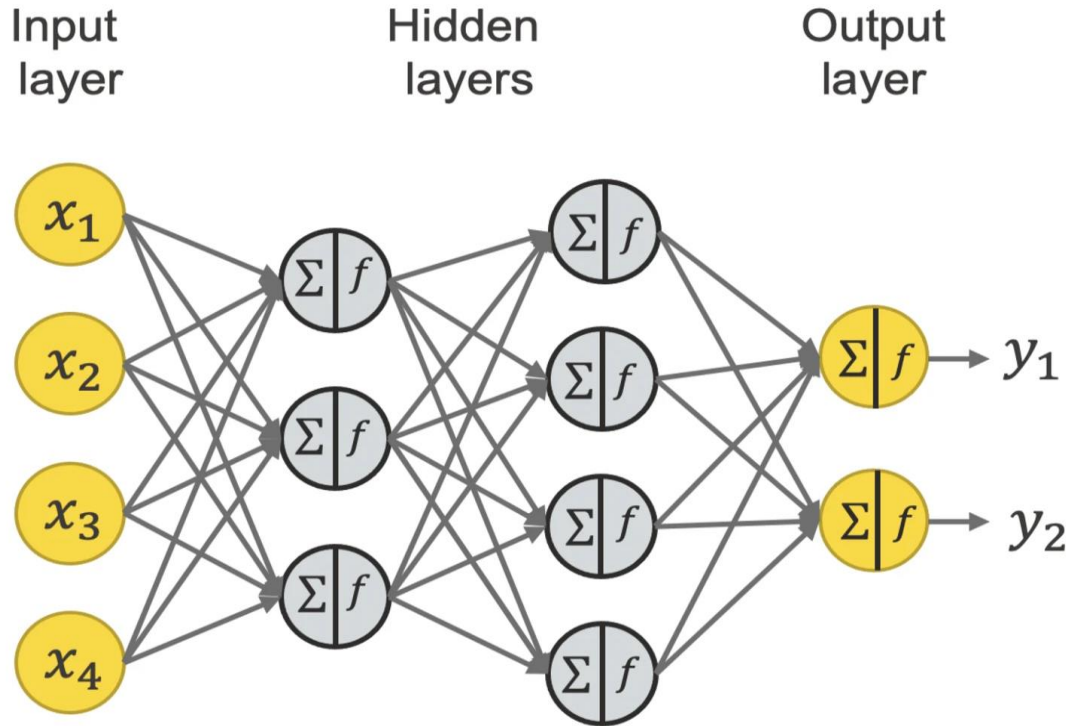
Nội Dung

1: Training deep learning models

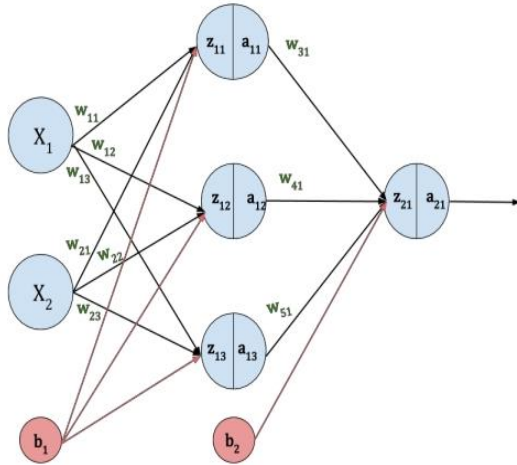
2: Training a better deep learning model.

1: Training deep learning models

Neural networks architecture



Forward Propagation



Forward propagation equation

$$z_{11} = x_1 w_{11} + x_2 w_{21} + b_1$$

$$z_{12} = x_1 w_{12} + x_2 w_{22} + b_1$$

$$z_{13} = x_1 w_{13} + x_2 w_{23} + b_1$$

$$a_{11} = \sigma(z_{11})$$

$$a_{12} = \sigma(z_{12})$$

$$a_{13} = \sigma(z_{13}) \quad (\sigma - \text{Activation function})$$

$$z_{21} = a_{11} w_{31} + a_{12} w_{41} + a_{13} w_{51} + b_2$$

$$a_{21} = \sigma(z_{21})$$

Vector Representation

$$\begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_1 \\ b_1 \end{bmatrix}$$

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$\begin{bmatrix} a_{11} \\ a_{12} \\ a_{13} \end{bmatrix} = \sigma \begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$z_{21} = \begin{bmatrix} w_{31} & w_{41} & w_{51} \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{12} \\ a_{13} \end{bmatrix} + b_2$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$a_{21} = \sigma(z_{21})$$

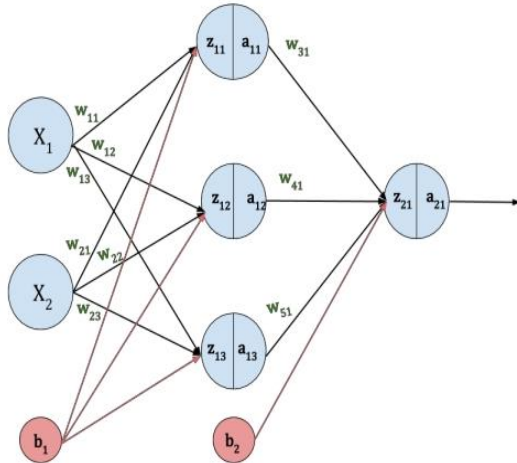
$$A^{[2]} = \sigma(Z^{[2]})$$

$$\begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix}$$

$$\text{và } \begin{bmatrix} w_{31} & w_{41} & w_{51} \end{bmatrix}$$

là bộ tham số mà mô hình này cần tìm được

Compute the Loss (Error)



Compute the Loss (Error)

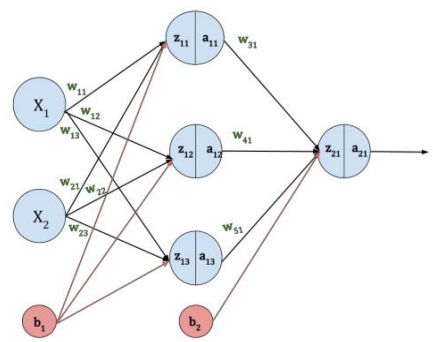
Loss function :-

$$J = -\frac{1}{m} \sum_{i=1}^m L(a^{[2](i)}, y^{(i)})$$

$$L(a^{[2]}, y) = -y \log a^{[2]} - (1 - y) \log(1 - a^{[2]})$$

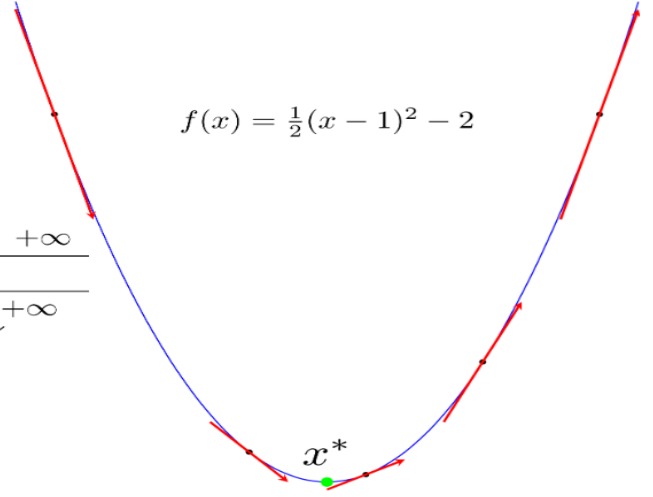
yêu cầu tìm giá trị bộ tham số sao Loss function là nhỏ nhất.

Gradient Descent



x	$-\infty$	1	$+\infty$
$f'(x)$	-	0	+
$f(x)$	$+\infty$	-2	$+\infty$

$$f(x) = \frac{1}{2}(x - 1)^2 - 2$$



Bước 1: Khởi tạo

- Starting point (điểm bắt đầu): x_0
- Learning rate (tỷ lệ học): α

Bước 2: Lặp lại

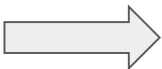
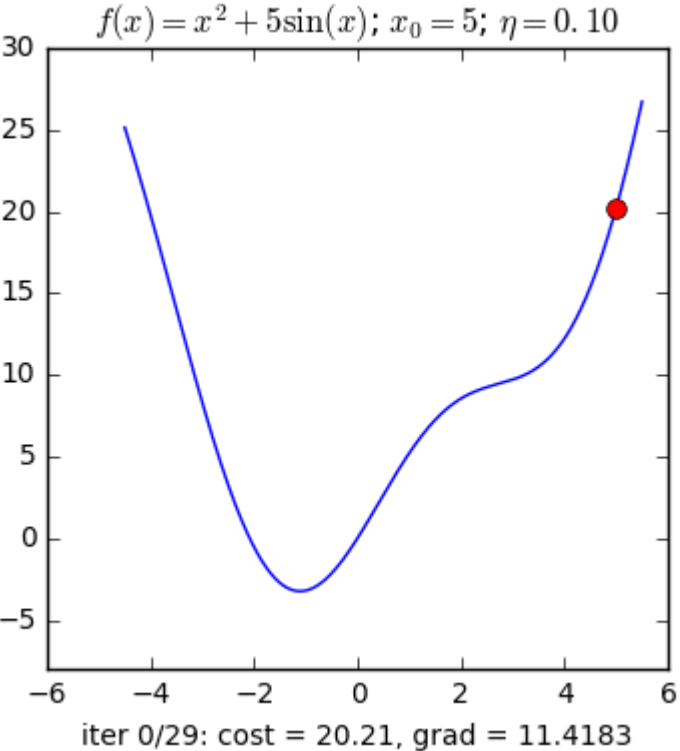
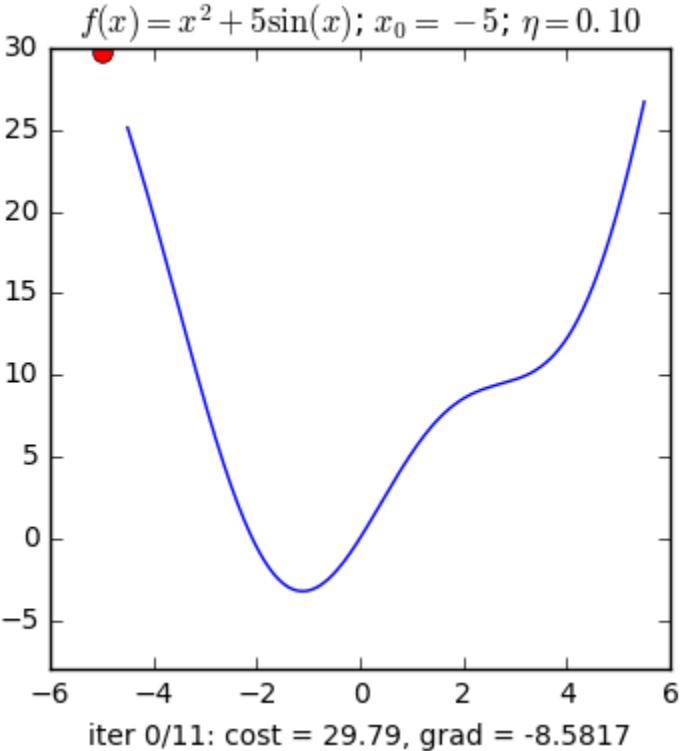
- Tính đạo hàm (Calculate the derivative): $f'(x)$ tại điểm hiện tại x
- Cập nhật điểm tiếp theo (Update the next point):

$$x = x - \alpha * f'(x)$$

Bước 3: Kiểm tra hội tụ

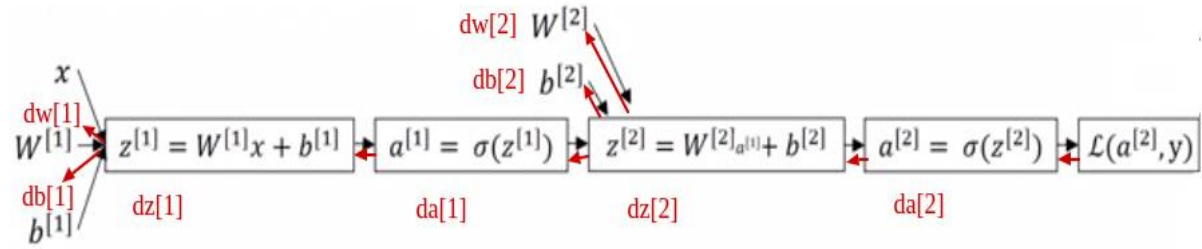
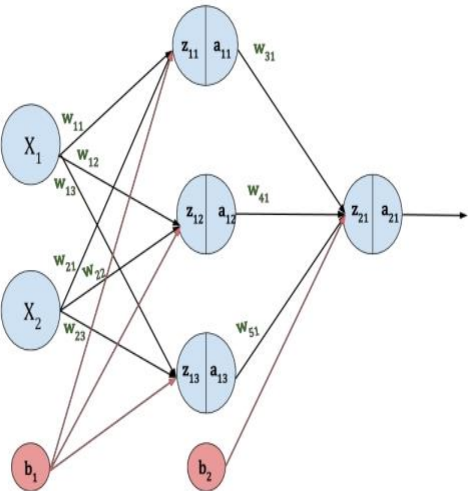
- So sánh giá trị $f(x)$ (Compare the value of $f(x)$): tại điểm hiện tại và điểm trước đó
- Chênh lệch (Difference): ϵ

Gradient Descent



learning rate

Back-Propagation Calculation



Chain Rule Formula

(i) $y = f(g(x))$

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$$

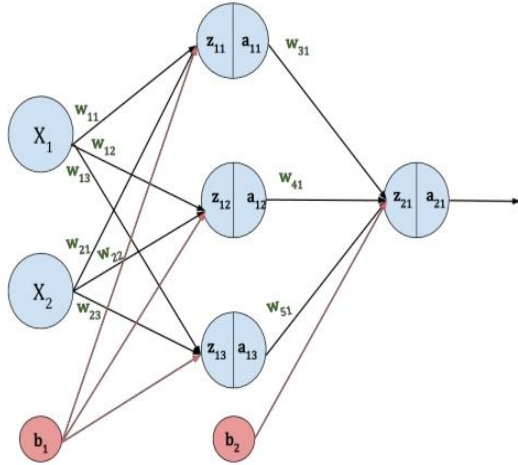
Outside's Function Derivative

Inside's Function Derivative

(ii) $y = f(t)$ and $x = g(t)$

$$\frac{dy}{dx} = \frac{dy}{dt} \cdot \frac{dt}{dx}$$

Weight Updating



$$W^{[1]} = W^{[1]} - \alpha \frac{\partial L}{\partial W^{[1]}}$$

$$b^{[1]} = b^{[1]} - \alpha \frac{\partial L}{\partial b^{[1]}}$$

$$W^{[2]} = W^{[2]} - \alpha \frac{\partial L}{\partial W^{[2]}}$$

$$b^{[2]} = b^{[2]} - \alpha \frac{\partial L}{\partial b^{[2]}}$$

(α - Learning Rate)

Mô hình hồi quy

```
# Xây dựng mô hình
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, input_shape=(1,), activation='relu'), # Tầng ẩn có
    tf.keras.layers.Dense(10, activation='relu'), # Tầng ẩn có 10 nơ-ron với hàm
    tf.keras.layers.Dense(1) # Tầng đầu ra với một nơ-ron
])

# Compile mô hình
model.compile(optimizer='adam', loss='mean_squared_error')

# Huấn luyện mô hình
model.fit(X_train, y_train, epochs=100)
```

Mô hình phân loại

```
# Xây dựng mô hình
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, input_shape=(2,), activation='relu'), # Tầng ẩn có 1
    tf.keras.layers.Dense(1, activation='sigmoid') # Tầng đầu ra với một nơ-ron và
])

# Compile mô hình
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Huấn luyện mô hình
model.fit(X_train, y_train, epochs=100)
```



Training a better
deep learning
model.



Một số vấn đề của mạng neuron

Vấn đề	Mô tả
Gradient biến mất hoặc bùng nổ	Gradient trở nên rất nhỏ hoặc quá lớn khi lan truyền ngược qua mạng dẫn tới quá trình cập nhật của mô hình không hiệu quả
Overfitting	Mạng nơ-ron quá tương quan với dữ liệu huấn luyện, dẫn đến hiệu suất kém trên dữ liệu mới.
High Variance	Mô hình quá nhạy cảm với các biến thể nhỏ trong dữ liệu huấn luyện, dẫn đến hiệu suất kém trên dữ liệu mới.
Tham số lớn	Trọng số trong mạng nơ-ron quá lớn, dẫn đến overfitting.
Dữ liệu thiếu	Dữ liệu huấn luyện có ít mẫu, dẫn đến overfitting.
Không đồng nhất về quy mô đặc trưng	Các đặc trưng của dữ liệu không có cùng độ lớn, ảnh hưởng đến hiệu suất mô hình.

Khởi tạo tham số

1. Glorot Initialization:

Đầu vào (fan_in) = 100

Đầu ra (fan_out) = 50

Phạm vi khởi tạo trọng số:

$$\text{Phạm vi} = \frac{2}{fan_in + fan_out} = \frac{2}{100 + 50} = \frac{2}{150} = 0.0133$$

Các trọng số được lấy mẫu từ phân phối đều trong phạm vi $[-0.0133, 0.0133]$.

2. He Initialization:

Đầu vào (fan_in) = 100

Phạm vi khởi tạo trọng số:

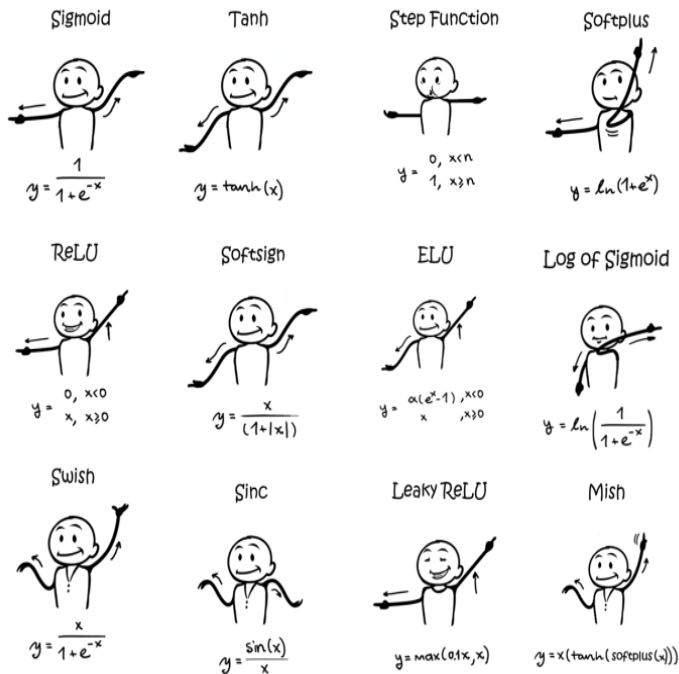
$$\text{Phạm vi} = \frac{2}{fan_in} = \frac{2}{100} = 0.02$$

Các trọng số được lấy mẫu từ phân phối đều trong phạm vi $[-0.02, 0.02]$.

Khởi tạo	Hàm kích hoạt	σ^2 (Phân phối chuẩn)
Glorot	None, tanh, logistic, softmax	$1/fan_{avg}$
He	ReLU and variants	$2/fan_{in}$
LeCun	SELU	$1/fan_{in}$

```
keras.layers.Dense(10, activation="relu", kernel_initializer="he_normal")
```

Activation Function



- Nhìn chung với mạng tuần tự thì Swish > SELU (scale elu) > ELU > Leaky relu > Relu > Tanh > Logistics.
- Nếu cấu trúc mạng không cho phép tự chuẩn hóa (deep and wide) thì elu hoạt động tốt hơn.
- Nếu quan trọng về thời gian dự đoán hãy dùng leaky relu,
- Mạng quá khớp RReLU
- tập huấn luyện lớn PReLU
- Dùng nhiều nhất ReLu

<https://www.linkedin.com/pulse/top-10-activation-functions-advantages-disadvantages-dash/>

BATCH NORM

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

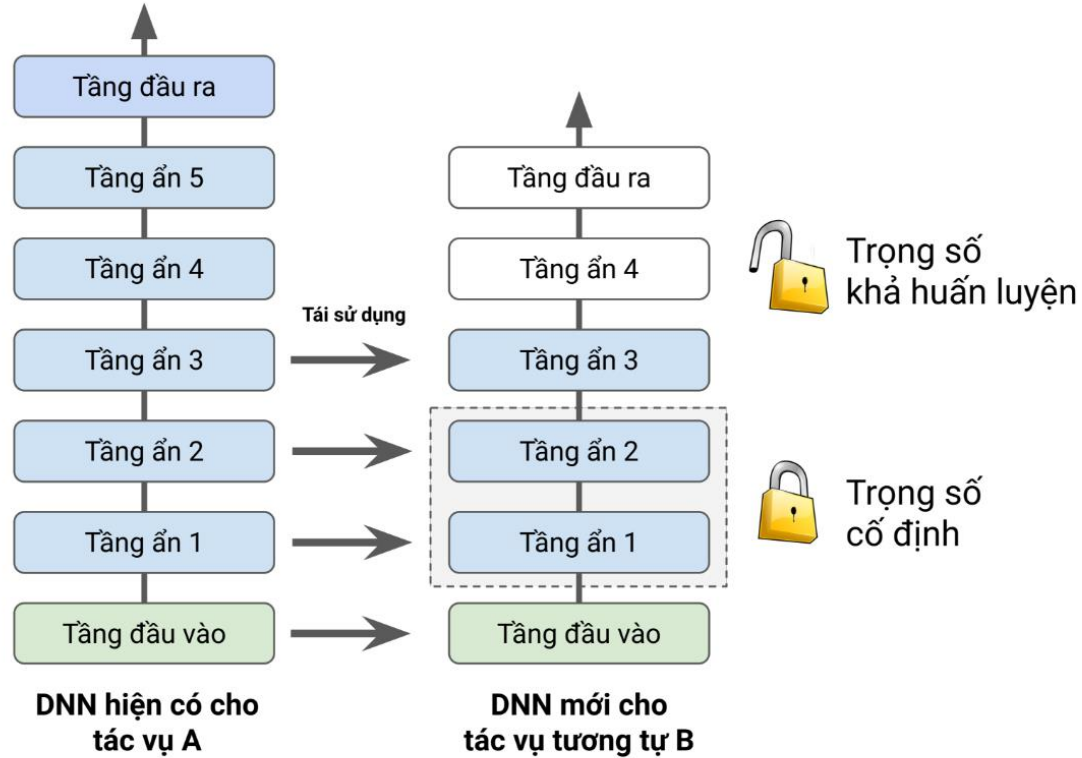
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

The simple equation behind batch normalization

Transfer Learning

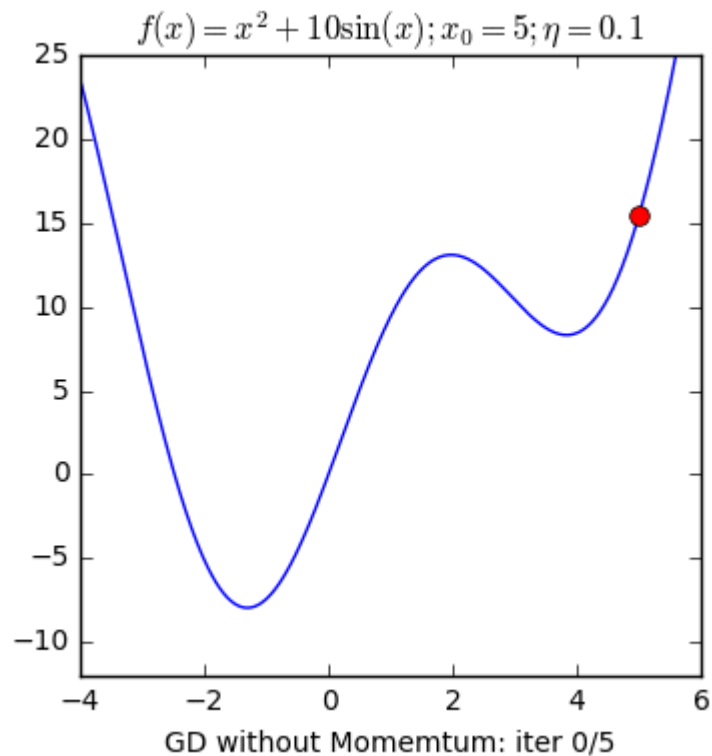


Hình 11.4. Tái sử dụng các tầng đã tiền huấn luyện

FASTER OPTIMIZERS

Stochastic Gradient Descent (SGD):

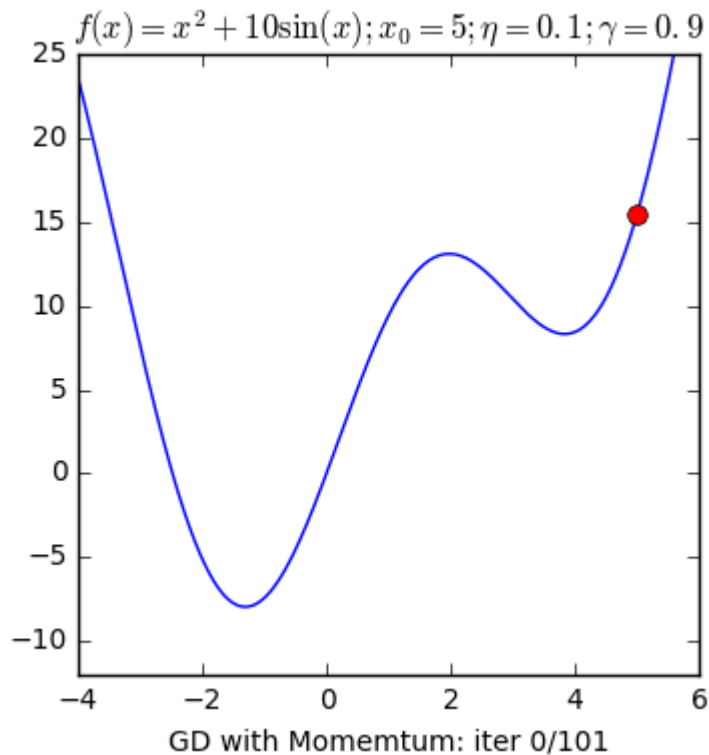
$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$



Momentum:

$$v_{t+1} = \gamma v_t + \eta \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$



FASTER

1. Stochastic Gradient Descent (SGD):

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

2. Momentum:

$$v_{t+1} = \gamma v_t + \eta \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

3. Nesterov Momentum:

$$v_{t+\frac{1}{2}} = \gamma v_t + \eta \nabla J(\theta_t - \gamma v_t)$$

$$\theta_{t+1} = \theta_t - v_{t+\frac{1}{2}}$$

4. AdaGrad:

$$g_{t,i} = \nabla_{\theta_i} J(\theta_t)$$

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{\sum_{r=1}^t (g_{r,i})^2 + \epsilon}} \cdot g_{t,i}$$

5. RMSprop:

$$g_{t,i} = \nabla_{\theta_i} J(\theta_t)$$

$$E[g^2]_{t,i} = \beta E[g^2]_{t-1,i} + (1 - \beta)(g_{t,i})^2$$

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{E[g^2]_{t,i} + \epsilon}} \cdot g_{t,i}$$

6. Adam (Adaptive Moment Estimation):

$$m_{t,i} = \beta_1 m_{t-1,i} + (1 - \beta_1) g_{t,i}$$

$$v_{t,i} = \beta_2 v_{t-1,i} + (1 - \beta_2) (g_{t,i})^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

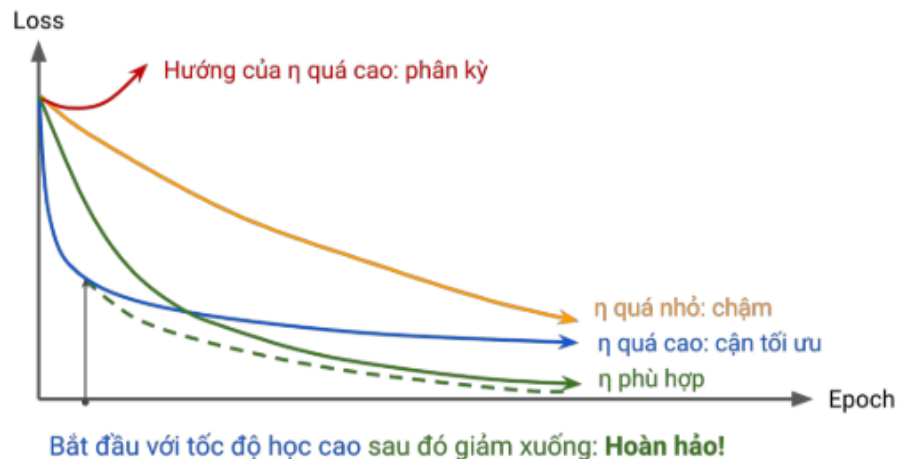
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \cdot \hat{m}_t$$

Bảng 11.2 so sánh tất cả các bộ tối ưu đã thảo luận (* là kém, ** là trung bình, và *** là tốt).

Bảng 11.2. So sánh các bộ tối ưu

Loại	Tốc độ hội tụ	σ^2 (Chất lượng hội tụ)
SGD	*	***
SGD(momentum=...)	**	***
SGD(momentum=..., nesterov=True)	**	***
Adagrad	***	* (dừng quá sớm)
RMSprop	***	** hoặc ***
Adam	***	** hoặc ***
Nadam	***	** hoặc ***
AdaMax	***	** hoặc ***

Learning rate schedule



✓ Power Scheduling

$$\text{learning_rate} = \text{initial_learning_rate} / (1 + \text{step} / \text{decay_steps})^{\text{power}}$$

- Keras uses power = 1.

✓ Exponential Scheduling

$$\text{learning_rate} = \text{initial_learning_rate} * \text{decay_rate}^{(\text{step} / \text{decay_steps})}$$

L1 vs L2 Regularization

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

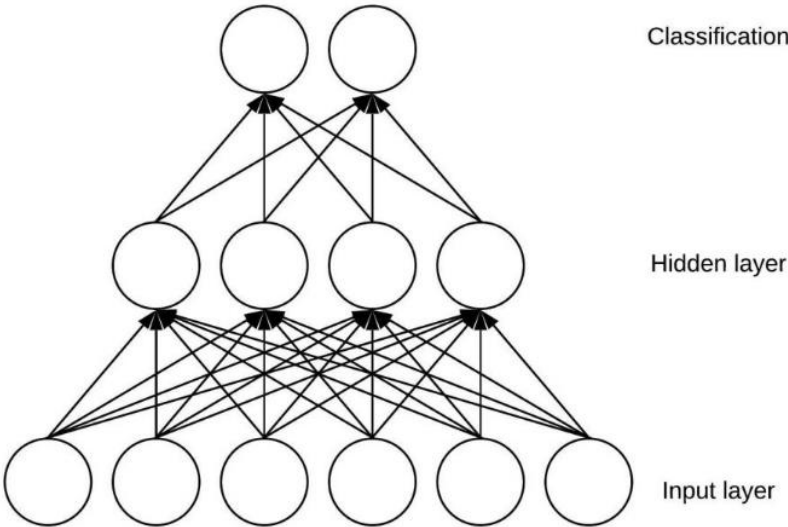
L2 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M W_j^2$$

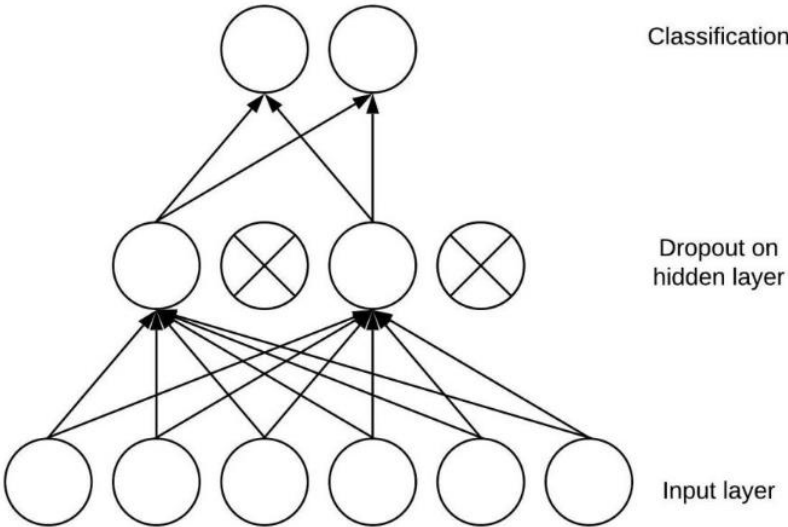
Loss function

Regularization
Term

DropOut



Without Dropout



With Dropout

